从提交到可见：原生套接字聊天系统的设计与演化

围绕顺序、所有权与真实交互的系统实验报告

Klee
Socket Chats Lab

Abstract

聊天系统的困难并非把文本送进套接字，而是让持久化、事件发布、本地缓存与用户操作对同一条消息给出一致解释。本文以 C++23 原生套接字实现的 Socket Chats Lab 为对象，讨论三进程架构如何组织这些边界，以及开发过程如何发现并修正边界间的错位。系统采用单反应器（reactor）、有界共享工作池、PostgreSQL 权威存储与按账户隔离的本地守护进程；通过覆盖执行至完成入队的房间门控（room gate）协调提交与发布，并将历史同步进度、房间元数据和前台可见位置分开建模。实验表明，消除错误等待与重复工作比普遍调整 TCP 参数更有效：本地完成唤醒将一次探测的中位响应从 15.532 ms 降至 0.126 ms，共享编码将匹配扇出工作负载的三次试验吞吐中位数提升至原来的 4.07 倍，批处理终端输入将 108 字符突发的完成时间从约 3.41 s 降至 26.82 ms。本文进一步分析契约驱动的并行开发、独立审阅与真实终端旅程如何共同推动模型简化。抽象状态机的 Lean 证明、SymPy 数值核算与复现细节置于附录。

Keywords

原生套接字, 事件驱动, 事务发布顺序, 终端交互, 工程协作

1 问题：消息经过了谁的边界？

从输入框到数据库，一条聊天消息会经历多个互不等价的时刻：用户提交、服务端接受、数据库提交、事件进入发布队列、对端接收，以及文本进入前台视口。把其中任何两个时刻合并，都会得到表面顺畅却语义含混的实现。例如，数据库已有记录不代表在线接收者已经看见；接收到最大消息 ID 也不代表此前的历史已同步；本地回显成功更不等于服务器已经确认。

本实验的目标是完成一个可以持续使用的终端聊天系统，而非孤立展示网络 API。系统包含服务端 chat-server、按账户运行的 chat-daemon 和交互前端 chat，支持租户内公共房间、持久消息、历史分页、未读统计与显式已读。约束为 C++23、操作系统原生套接字和不引入第三方网络框架；PostgreSQL 通过 libpq 接入，终端组件采用固定版本的 FTXUI。默认部署是本机明文回环连接，远程访问使用 SSH 隧道。这个范围使我们可以集中讨论单服务进程内的持久化与交互一致性，而不将多副本共识带来的问题混入同一模型。

核心判断。. 可靠性来自让语义边界与状态所有权对齐，而非不断为异常路径添加补丁。本文围绕三个相互关联的问题展开：

(1) 反应器、工作线程与数据库应当怎样分工，才能同时保持会话顺序、房间顺序和有限资源占用？
(2) 断线、历史分页和终端渲染应当怎样共享消息身份，又不错误共享进度？
(3) 当测试通过而用户体验仍然失败时，开发组织与验证方法应当如何改变？

这三个问题最终落在同一原则上：*共享稳定事实，分离生命周期与进度。*稳定消息身份可以跨缓存、历史和直播复用；套接字存活状态不能决定已提交事件是否发布；房间最大 ID 不能替代历史抓取游标；本地守护进程持续在线也不能代替用户的已读意图。

与既有工作的关系。. 分阶段事件驱动架构（Staged Event-Driven Architecture, SEDA）用显式队列表达负载与调度边界 [6]。本系统采用其有界排队思路，但保留一个反应器与一个共享工作池，不为每个业务阶段建立独立线程池。基于性质的测试（property-based testing）将不变量转成生成式输入上的检查 [1]；我们将其用于字节流、缓存和游标，再以进程及真实终端测试补上环境语义。IronFleet 展示了实现级机器验证的深度 [2]；本文则对关键顺序抽象作局部机器证明，并用可观察旅程检验实现与抽象的连接。近期 Legolas 通过抽象状态引导局部故障注入 [7]，与本实验在提交后断线、未确认发送和历史页取消等边界布置判别性测试的方向相呼应。

2 围绕语义边界组织系统

系统由服务端、账户守护进程与终端客户端构成。三者的划分并非按界面层次机械拆分，而是对应三种寿命：服务端管理权威事实，守护进程维持账户连接与可恢复的本地视图，终端只管理一次交互会话。由此，关闭窗口不再意味着停止接收消息，恢复界面也不必重新解释网络协议。设计的核心问题随之收敛为：哪些状态必须串行，哪些工作可以并行，以及何时能够向用户宣告一次操作已经成立。

2.1 以所有权约束并发，而非以线程划分连接

服务端由单个反应器独占套接字、增量解码器与收发队列，将 SQL 和领域操作交给全局有界工作池。工作线程只返回操作结果，不直接写入连接。该结构借鉴分阶段事件驱动架构对显式队列与过载控制的强调 [6]，但不为每类操作建立独立执行级：对这一规模的聊天负载，单一工作池已经能够隔离数据库阻塞，进一步分级反而增加调度与状态迁移。

每个会话最多提交一个在途任务；后续命令保留在该会话的先进先出队列中。于是，认证、加入房间、发送与读取天然保持客户端提交顺序，而不同连接仍可并行执行。Linux 使用水平触发的 `epoll`，其余平台使用 `poll`/`WSAPoll`；选择水平触发是为了避免把边沿重置与必须彻底排空输入的约束传播到业务层 [3]。一次就绪事件的处理量受到限制，慢接收者则由独立输出预算约束。

所有权也决定退出语义。收到 TCP FIN 仅表示输入结束：已经完整接收的命令继续执行，回复排空后再关闭套接字。否则，“发送后关闭写端”会被错误解释为取消已提交工作。工作完成通过原生非阻塞套接字唤醒反应器，而不是等待周期轮询；通知缓冲区已满意味着已有待读通知，无须增加重试线程。广播则只编码、压缩一次，各接收者共享不可变字节缓冲区并保有自己的写入偏移。资源核算仍按每个接收者的逻辑字节计费，避免共享存储意外放宽慢客户端限制。

2.2 提交与发布是同一个顺序问题

数据库事务解决持久化，却不会自动保证网络观察顺序。若较小编号的消息已经提交、但其工作线程尚未发布结果，另一个

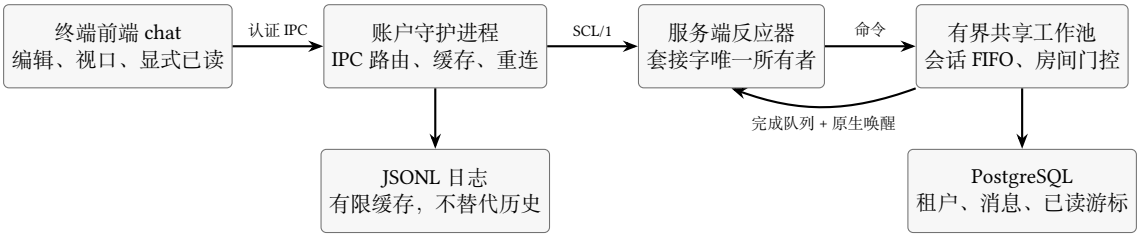


Figure 1: 状态所有权而非功能列表决定进程边界。数据库提交之后，工作线程只发布完成对象；所有客户端套接字写入仍由反应器完成。守护进程独立于某一个终端窗口存活。

线程就可能先广播较大编号。客户端据此推进历史或已读游标，便可能跨过尚未看到的消息。这里缺失的不是另一条 SQL 约束，而是从事务执行到结果入队之间的连续顺序。

系统以租户和房间为键映射到固定的 256 个互斥条带，形成条带锁（striped locking）；相关操作从执行开始持锁，直至完成结果进入反应器的有序完成队列。数据库房间行锁约束事务内部的发送与读取；进程内条带则把提交顺序延伸到事件发布顺序。固定条带不随房间数量增长，代价是哈希碰撞可能串行化无关房间。热点房间本来就需要共享顺序，因此首先保留这一简单结构，而非引入可回收锁表及其额外寿命协议。不同房间不提供全局事件顺序。

PostgreSQL 是权威存储 [4]。发送必须提交后才生成成功确认或广播；租户、用户和发送随机标识组成唯一键，使原内容重试返回原消息编号而不重复广播，改写内容则产生冲突。身份取自认证会话，而非普通命令字段。数据库连接以独占租约交给工作线程，避免并发使用同一连接。连接故障时不自动重放事务，因为“未收到确认”无法区分“未提交”与“已提交但回复丢失”。这一不确定性必须保留到上层，而不能用盲目重试掩盖。

2.3 把本地状态绑定到账户，而非窗口

守护进程把远端凭据与本地 IPC 凭据分离，以账户身份文件和操作系统独占锁约束状态目录。同一目录既不能并发写入，也不能被另一账户静默接管。端点文件原子替换，消息日志不记录认证帧；POSIX 私有权限与 Windows 当前用户专属访问控制分别落实平台边界。多个终端的请求编号在转发时重新映射，响应因而不会因编号碰撞交付给错误窗口。

本地 JSONL 日志是有界缓存，而非第二个数据库：它支持离线浏览、容忍损坏记录，并通过压缩替换控制体积，但不承担提交持久性。重连采用退避，既有终端继续浏览；补齐历史由前台根据独立的已获取游标完成。房间公告的最新编号、实时收到的最大编号与历史分页进度不能合并，否则实时事件会把分页起点推过尚未拉取的区间。

界面发送采用乐观呈现，再按消息编号或作者与随机标识合并确认、广播和历史。尚未转发的离线失败与已经转发但结果丢失必须区分：前者可明确失败，后者显示交付未知，不自动重发。这样的状态划分使用户既能立即看到输入结果，也不会把本地回显误认作数据库承诺。

2.4 已读是一项界面语义，而非接收副作用

守护进程收到消息、缓存重放以及历史查询均不推进已读状态。只有前台显式操作才能提交游标，且游标来自当前布局中确实进入视口的消息；历史同步期间暂缓标记。数据库再保证游标单调且不超过房间已提交最大编号。这里的“已读”准确含义是用户确认已显示的消息前缀，而非系统推断用户阅读了每一个字符。将该语义放在布局之后计算，才能使滚动、换行、帮助遮罩和窗口大小真正参与正确性。

终端层使用固定版本的 FTXUI 管理输入、布局和屏幕生命周期 [5]，网络仍使用原生套接字。独立有界 I/O 线程合并通知到界面事件循环，使输入节奏不再依赖网络等待。消息正文先消除不可信控制序列；界面只为视口内容建立显示节点，并以有界换行缓存减少长历史重排。与此同时，脚本用纯文本模式保留原有命令契约，使交互升级不破坏既有用户。

这些选择明确界定了系统的伸展方向：单进程承担实时广播，跨进程共享 PostgreSQL 尚不等于分布式订阅；房间未读统计仍需访问历史，持续增长的历史将推动聚合与保留策略；Unicode 宽度估计也不替代完整字素塑形。当前优先保证账户隔离、消息顺序与交互连续性，后续扩展应维护这些已有语义，而不是仅替换底层组件。

3 开发组织：让反例进入设计

本项目将开发组织视为系统设计的一部分：并行工作的边界必须与状态所有权一致，而验证必须能够推翻当前实现。最初的共享契约先固定帧格式、认证身份、提交后发布、逐会话先进先出等不变量，再划分协议与套接字、服务端存储、客户端及集成责任。这样，工作者交换的是明确的接口与约束，而不是对彼此代码的隐含猜测。终端重构沿用同一原则：依赖与原生输入机制、纯视图构造、会话传输与应用状态、真实终端测量分别归属不同任务；涉及接口的变更先回到共享契约，再进入集成。分工不是尽可能多地同时修改文件，而是减少需要共同理解和共同修改的状态。

文档是可修订的共享黑板。. 架构契约、存储语义、终端决策与验证记录构成持久的共享黑板（shared blackboard）。例如，终端方案从自建输入与绘制引擎转向固定版本的 FTXUI 后，旧草案保留为决策轨迹，却明确退出当前实现要求；否则，历史方案容易成为并行任务之间相互矛盾的隐性规范。评审记录同样区分初始发现、后续修正与执行证据：一条已修复的历史缺陷仍有解释价值，但不能被继续当作当前结论。黑板由此承担两项职责：使后来者恢复设计理由，并使团队能够撤销已经失效的认识。

独立评审寻找跨边界反例。. 最有价值的缺陷并非局部语法错误，而是两个分别合理的模块组合后破坏了整体语义。R1 发现，发起连接关闭后丢弃整个工作完成项，会同时丢弃已经提交、应向其他连接广播的消息；修复将请求者生命周期与领域事件发布解耦。R2 则构造了另一种调度：较小编号已经提交，但其工作线程尚未入队，较大编号先发布；接收端随后提交较大已读游标，便可能将尚未显示的消息标记为已读。这个反例

说明，逐会话顺序与数据库内的房间锁不足以保证跨发送者的显示顺序，房间级顺序边界必须延伸到完成项入队。评审的产物因此不只是问题清单，也是对所有权边界的重新刻画。

客户端评审继续沿着数据到用户判断的路径追踪语义。T1 揭示房间最大编号、实时消息编号和连续取回的历史游标不能共用一个变量；元数据声称存在某条消息，不等于客户端已经取得此前的历史。T4 发现，已转发但未收到确认的发送被表示为确定失败，会诱导用户用新随机标识重发；交付不确定性必须跨守护进程与界面保持一致。T5 则暴露了有界缓存与历史浏览的冲突：若始终淘汰最旧消息，刚取回的旧历史会立即消失。改为历史窗口后，还需处理返回最新时旧分页仍在途的取消竞态；后续响应必须服从当前浏览意图，而不能凭旧请求上下文重新启用历史淘汰策略。

用户旅程决定证据是否相关。. 用户报告 Windows 客户端逐字迟滞且对话难以阅读时，既有管道测试的成功并不能反驳这一观察：它们绕过了原生键盘、终端布局与输出刷新。验证因而转向真实伪终端（pseudoterminal, PTY）与 Windows 伪控制台（ConPTY），通过独立协议端点确认实际发送，并检查当前屏幕、编辑中的草稿、重连和终端恢复，而非仅搜索累计输出。性质测试（property-based testing）用于探索字节流分片、UTF-8 和游标状态空间 [1]；确定性旅程则固定跨组件反例。历史回归先让旧二进制停在第 200 条而非第 300 条，再验证修复；取消竞态则扣住旧页结束响应，执行返回最新，随后释放响应，检查旧请求不会复活。这样的失败具有设计指向性，而非一次笼统超时。

测试设施本身也必须接受反例。. macOS 执行暴露了另一个边界：测试线程等待协议端点时不读取 PTY，可能使同步屏幕输出阻塞。修正是持续排空输出，而非修改应用输入解析。随后，控制会话退出后的终端属性检查改为由仍存活的会话启动器在挂断前记录；对于内核自动设置的输入重处理状态，则在前后快照中一致地执行不消耗输入的就绪查询，仍要求完整属性相等。负向探针保留错误的回显设置，确认恢复检查依然能够拒绝缺陷。由此得到的组织经验不是未经对照的生产率提升，而是一条可检查的闭环：共享契约约束并行实现，独立反例修订模型，真实旅程检验用户可见结果，文档保存下一轮工作所需的理由。

4 实验：定位等待，消除重复工作

实验遵循同一节奏：先由用户旅程确定问题，再以代码路径提出机制假设，最后选取能区分竞争解释的对照。这里报告项目已留存的 2026 年 10 月实验；其环境、原始记录位置与执行命令集中列于附录 A。不同工作负载回答不同问题，不混合计算一个综合“性能提升”。

4.1 等待策略比轮询周期更重要

最初服务端通过周期性检查接收工作完成结果。即便业务操作很轻，完成对象仍需等待下一次轮询。Windows 本地、单客户端、两工作线程的 100 次顺序 ping 探测中，旧路径中位延迟为 15.532 ms。改为先入完成队列、再写入原生非阻塞回环唤醒套接字后，中位延迟为 0.126 ms，p95 从 16.163 ms 降至 0.222 ms。这里的改变不是缩短超时，也不是提高系统计时器精度，而是让“存在待处理工作”直接成为就绪事件。

相同问题在终端出现了另一个版本。旧交互循环先等待网络 25 ms，再读取一个 UTF-16 按键，因而粘贴和缓冲输入被错误地序列化为逐字符等待。替换为 FTXUI 事件批处理后，真实

Table 1: 不同路径的配对延迟实验。各行只在本行环境内比较；突发值为三次完成时间的中位数，单键值为 78 个观测的 p50。

测量路径	原路径 (ms)	改进路径 (ms)
服务端完成通知：ping p50	15.532	0.126
服务端完成通知：ping p95	16.163	0.222
终端：ASCII 单键 p50	30.844	30.642
终端：108 字符 ASCII 突发	3410.650	26.817
终端：76 码点中/emoji 突发	2410.523	28.062

Table 2: 共享编码的三次匹配扇出试验。每次 1600 条消息；改动仅为编码缓冲区复用。

	ACK/s		发送至 ACK p50 (ms)	
试验	逐接收者	共享	逐接收者	共享
1	659.50	2604.69	24.086	5.767
2	647.36	2652.47	24.639	5.492
3	651.64	2694.16	24.466	5.568

Windows 伪控制台（ConPTY）中的 108 字符 ASCII 突发，中位完成时间从 3410.650 ms 降为 26.817 ms；76 码点中文/emoji 突发从 2410.523 ms 降为 28.062 ms（表 1）。单键 p50 仍约 30.6 ms，说明主要收益是移除与字符数成正比的额外等待，而非改变所有输入的固定显示延迟。

终端实验同时要求独立 Bob 协议客户端观察到精确正文，避免把本地回显当作发送成功。旧、新客户端都完成了共同工作负载的九条消息；新路径还通过逐键中文/emoji 和光标编辑，独立对端共验证十三条消息。延迟终点是当前屏幕投影出现预期草稿，不是输出日志中曾出现某个片段。读线程持续排空输出，测试宿主响应 Win32 输入模式协商，避免测量管道误入普通输入路径。

4.2 扇出优化：共享字节，而非共享可变进度

每条房间事件在各接收者处拥有相同的编码结果。原实现却为每个接收者重新序列化与压缩，包括重复初始化 LZ 编码表。改进后每事件只编码一次，各输出队列共享不可变字节缓冲区，同时保留各自写偏移。背压仍按每接收者的逻辑字节计费，不因底层共享而放松慢客户端限制。

在 Windows GCC Debug、内存后端、四工作线程、16 客户端各发送 100 条 256 字节正文的匹配 A/B 中，三次试验的吞吐中位数从 651.64 ACK/s 提升至 2652.47 ACK/s，发送至 ACK 的试验 p50 中位数从 24.466 ms 降至 5.568 ms（表 2）。这一结果对应移除重复计算的机制，也保持原有协议与压缩格式。测量包含 Python 接收端和高可压缩文本，是本地扇出工作负载的结果，不作为 PostgreSQL 容量估计。

4.3 一个被放弃的解释：普遍启用 TCP_NODELAY

对于小消息高延迟，Nagle 与延迟 ACK 的交互是自然假设，但机制存在并不意味着它解释当前观测。早期保留二进制与启用 TCP_NODELAY 的新二进制比较，看似显示明显差异；然而在同一当前源码上撤回选项后，性能没有回到早期水平。这个反向对照揭示了源码身份混杂。当前源码的启用组与撤回组 p50 分别约 24.5–25.1 ms 和 24.3–24.9 ms，没有带来值得保留的改善。

因此，系统只在完成唤醒通道保留窄范围 TCP_NODELAY，而不对所有业务连接施加同一策略。后续共享编码的匹配 A/B 则直接检验了重复压缩这一解释。这个过程的价值不是得到一个“最佳 TCP 参数”，而是将待优化对象从传输层猜测重新定位到应用内的工作放大。

4.4 正确性覆盖：从字节流到真实终端

性质测试覆盖分片/合并等价性、UTF-8 与有界解压、JSONL 往返和游标转移；进程测试覆盖认证、租户隔离、双 IPC 客户端的碰撞请求 ID、FIN 后命令排空及断线重连。240 个并发同房间事件由独立观察者检查严格 ID 顺序；20 次发送后立即 FIN 与 32 命令合并后 FIN，检查已提交消息不会因请求者生命周期而消失。

真实终端测试进一步覆盖异步收信时编辑、中文/emoji、窄屏、滚动后显式已读、断线草稿、历史窗口往返与退出恢复。附录 A 所标识的跨平台执行在 Windows、Linux、macOS、强制 poll 及 ASan/UBSan 配置完成相应的 13 项测试；PostgreSQL 专用旅程还验证服务重启后的字段一致性，以及连接池会话终止后的原进程恢复。与吞吐数字相比，这些旅程更直接刻画系统是否完成了用户想做的事情。

5 讨论：把一致性推进到交互边界

单房间串行化是一种明确的交换。. 256 条锁带把活跃房间映射到有限同步对象，避免动态锁表的创建、回收和生命周期问题。碰撞会使无关房间暂时串行，热房间等待也会占据共享工作线程。这是当前模型为简单顺序语义付出的代价。下一步有价值的研究不是直接增加线程数，而是在热点房间与普通房间混合负载下测量排队时间；若跨房间尾延迟显著恶化，再考虑按房间就绪队列调度，让等待发生在队列中而非占用工作线程。公平性优化必须保持执行至完成入队这一原子顺序边界。

持久化日志不自动成为发布日志。. 当前数据库权威历史与内存完成队列各司其职。运行中请求者断开不会丢弃已提交广播，但服务进程若在提交与完成入队之间崩溃，在线通知仍可能缺失，之后通过历史补齐。若要扩展为多服务实例或要求崩溃后自动补发，需要将待发布事实与消息写入同一持久事务，并定义发布重试和消费去重。此时问题不再是扩大现有锁，而是引入持久发布日志；跨实例实时扇出也应成为显式协议能力，而不是假定 PostgreSQL 行锁能够提供它。

可见位置是交互事实，不是传输事实。. 显式已读将确认权留给前台用户，视口给出本次确认的上界；这一游标仍表示房间前缀，而非逐消息注意力证明。初始仅同步最近窗口、用户跳转或折叠内容时，系统不能据此推断每个历史字符都被阅读。更细粒度的可见区间或逐消息确认会改变存储成本、未读定义与 UI 负担。当前选择保留熟悉的前缀游标语义，并严格区分“收到”“同步过”和“允许显式确认”的状态。

边界应当成为类型与协议的一部分。. 对已转发后断开的发送返回 delivery_unknown，而对尚未入队的离线发送返回 offline，比在同一个错误字符串里解释两种情况更可靠。不可变广播缓冲区与独立写偏移、独立历史抓取游标与元数据最大值，也都是同一种设计：先使状态表达正确，再让普通控制流承担原来的特殊情况。后续演化应继续保留现有帧含义和普通 CLI 行为，在兼容字段或明确版本边界上扩展。

6 结论

本实验将一个原生套接字聊天系统的主要困难归结为跨边界语义的组织：会话 FIFO 不等于房间顺序，数据库提交不等于事件发布，接收最大值不等于历史进度，本地回显不等于成功发送。通过明确所有权、对齐发布门控、拆分游标以及采用真实终端旅程，系统在保持协议兼容的同时消除了多类错位。性能改进也来自同一方向：用工作就绪替代周期等待，用不可变共享字节替代重复编码，用成熟终端批处理替代逐字符网络轮询。开发文档、独立审阅和反例回归共同把这些原则转化成可以持续维护的工程结构。

References

[1] Koen Claessen and John Hughes. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*. ACM, 268–279. doi:10.1145/351240.351266

[2] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jay R. Lorch, Bryan Parno, Justine R. Stephenson, Srinath Setty, and Brian Zill. 2015. IronFleet: Proving Practical Distributed Systems Correct. In *Proceedings of the 25th Symposium on Operating Systems Principles*. ACM, 1–17. doi:10.1145/2815400.2815428

[3] Michael Kerrisk. 2025. *epoll(7): Linux Manual Page*. https://man7.org/linux/man-pages/man7/epoll.7.html

[4] PostgreSQL Global Development Group. 2026. *PostgreSQL 18 Documentation: Explicit Locking*. https://www.postgresql.org/docs/18/explicit-locking.html

[5] Arthur Sonzogni and contributors. 2026. *FTXUI: C++ Functional Terminal User Interface, Version 7.0.3*. https://github.com/ArthurSonzogni/FTXUI/tree/v7.0.3

[6] Matt Welsh, David Culler, and Eric Brewer. 2001. SEDA: An Architecture for Well-Conditioned, Scalable Internet Services. In *Proceedings of the Eighteenth ACM Symposium on Operating Systems Principles*. ACM, 230–243. doi:10.1145/502034.502057

[7] Haoze Wu, Jia Pan, and Peng Huang. 2024. Efficient Exposure of Partial Failure Bugs in Distributed Systems with Inferred Abstract States. In *21st USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, 1267–1283. https://www.usenix.org/conference/nsdi24/presentation/wu-haoze

A 复现环境、证据与工程细节

本报告对应仓库 socket-chats-lab，撰写基线为 034ace8。正文历史实验来自项目保留的执行记录，不将本文排版时刻视作全部性能实验的重跑时刻。设计核对使用当前代码；原始性能数据、前后源文件与二进制保存在本地 .cache。仓库内 report/evidence.md 记录数值来源与代码映射，docs/ 保存可长期复用的实验说明。

A.1 构建与完整用户旅程

应用要求 CMake 3.25+、C++23 编译器和 Python 3.10+。默认构建启用 libpq；便携构建显式关闭数据库依赖，运行内存后端。FTXUI 7.0.3 由 FetchContent 从固定源码归档取得，SHA-256 为：

```
e7c62ffe19009759821b4f0f8df7f2a6fb83784c3a9f1477d81f56d3ee723c88
```

Windows 输入补丁由 scripts/patch_ftxui.py 施加，按固定源上下文校验；应用网络仍完全使用原生套接字。构建产物位于仓库 .cache/build，不提交依赖源码、数据库凭据或守护状态。

```
python scripts/dev.py build --portable
python scripts/dev.py test --portable
python scripts/dev.py demo --portable
```

双用户旅程为：启动两个同租户账户的守护进程与终端；发送中文和 emoji；切换房间、检查未读和分页；显式标记当前视口已读；停止上游后继续编辑；恢复上游并确认草稿未自动发送；退出并检查终端模式恢复。普通 --plain 路径用于脚本兼容性，不能替代该交互旅程。

PostgreSQL 复现使用新的专用角色与数据库，按照 README.md 的部署步骤配置 CHAT_DB_DSN 与预置令牌。tests/journey.py--help 和 tests/postgres_recovery.py--help 给出所需服务端、守护进程及客户端参数。前者以 --postgres 验证服务重启前后消息字段与房间元数据；后者会终止目标数据库用户的池连接，必须使用专用测试数据库。凭据只经环境或受保护配置传入，不写入论文或版本库。

A.2 实验配置与原始数据

完成唤醒。. Windows 11、GCC 16.2 Debug、内存后端、两工作线程、回环 TCP；预热十次，顺序测量一百次。旧路径为 10 ms 完成轮询，新路径为原生套接字唤醒。docs/storage.md 记录首次新路径中位值 0.12635 ms、p95 0.2218 ms，后续核算版本为 0.12275/0.2025 ms。正文统一使用首次配对值。

```
python tests/server_latency.py --server SERVER --iterations 100
```

广播共享编码。. Windows 11 build 26200，MinGW GCC 16 C++23 Debug，四工作线程，内存后端，16 个 Python 线程客户端使用同一租户/用户，每客户端一条在途发送、各发送 100 条消息，同时接收租户广播。正文为 256 UTF-8 字节，带中文/emoji 前缀和重复字符，具有较高可压缩性。接收端 Python 解压和运行时竞争包含在测量中；ACK/s 的分母包括客户端线程启动及认证时间，不是仅截取稳定期的服务端吞吐。三次先前/共享交替运行，唯一 A/B 源差异为广播编码复用：

```
python scripts/benchmark.py --server SERVER --clients 16 --messages 100 --bytes
      256 --threads 4 --output .cache/server/TRIAL.json
```

基线源文件与可执行程序为 .cache/server/broadcast-baseline-main.cpp 和 broadcast-baseline.exe；六份记录为该目录下 broadcast-before-1.json 至 broadcast-after-3.json。表 2 保留每次测量，不把三次试验中位数当作单次原始样本的分位数。

交互输入。. Windows 11 build 26200 x64，Intel family 6 model 154、20 逻辑 CPU，Python 3.14.6，两版客户端均为 MSVC Debug；服务端和守护进程保持相同。ConPTY 为 140 列、36 行，各三轮 ASCII 单键、108 字符 ASCII 突发、76 码点中文/emoji 突发。测量从输入管道写入前计时，到持续读取线程重建的当前屏幕出现目标草稿；它不包含实体键盘、显示器或 IME 候选窗时延。

```
python scripts/terminal_probe.py --server SERVER --daemon DAEMON --client CLIENT
       --output .cache/report/conpty --editing
```

历史基线位于 .cache/tui-baseline/conpty-authoritative/result.json，新客户端为 .cache/tui-current/final-color/result.json；精确程序哈希和宿主协商方法见 docs/interactive-latency.md。基线额外逐键 Unicode 阶段曾停顿，故共同九条消息与新版扩展十三条消息分开报告。所有子进程由隔离夹具清理。

TCP 参数反向对照。. 原始记录为 .cache/benchmark-nodelay-1.json 至 benchmark-reverted-3.json，以及更早的 benchmark-nagle-*.json。前两组当前源码比较用于判断选项收益；更早程序只用于展示控制组混杂，不用于归因。完整环境与决定见 docs/performance-network.md。

A.3 性质与故障测试的可追踪入口

Table 3: 核心不变量与实现级检查。种子与次数对应当前测试源码。

性质/旅程	源码与检查规模
编码、分片与合并	tests/core_tests.cpp；种子 42，600 组往返/分片与 1000 组变异
存储及游标	tests/server_storage.cpp；种子 74821，400 次游标转移，并发重复发送与冲突
请求者断开后发布	tests/server_publication.py；20 次 FIN、32 条合并命令、240 条同房间并发事件
缓存与重连	tests/client_tests.cpp、tests/client_journey.py；JSONL、账户目录、离线/重连及权限
真实终端与历史窗口	tests/terminal_journey.py、tests/tui_state_journey.py、tests/tui_window_journey.py

已执行的 FTXUI 跨平台矩阵对应提交 eb24fa738f5eccf7785db6fb940a09f004171609，公开执行记录为GitHub Actions 39286126175；Windows/MSVC、Ubuntu/GCC、macOS ARM64/AppleClang、PostgreSQL/强制 poll、ASan/UBSan 均完成各自 13 项测试。后续配色与最终代码的执行记录另见 docs/validation.md 和 docs/tui-color-system.md。历史性能试验不与这些 CI 结果混为同一环境。

POSIX 强制 poll 与 sanitizer 可按项目预设分别执行：

```
cmake --preset sanitize
cmake --build --preset sanitize
ctest --preset sanitize
cmake -S . -B .cache/build/poll -DCHAT_WITH_POSTGRES=OFF -DCHAT_FORCE_POLL=ON
cmake --build .cache/build/poll --parallel
ctest --test-dir .cache/build/poll --output-on-failure
```

A.4 协议与资源界限

SCL/1 使用 20 字节定长头，包含魔数、版本、类型、标志、请求 ID、线上长度和解压长度；字段为数量加长度前缀的 UTF-8

字符串。原始及线上负载均不超过 1 MiB，字段数不超过 128。身份来自认证会话，普通消息不接收可信租户覆盖。认证帧不压缩，每帧独立字典，解压同时限制声明长度与实际输出。

当前服务端允许最多 1024 连接，等待工作队列为 1024 任务，每会话最多 64 待处理命令和约 2 MiB 输入估算；全局输入、完成队列和输出各有 64 MiB 核算限额，单客户端输出为 4 MiB。历史每页最多 200 条且按字节提前结束。该核算不等于整个进程 RSS：对象开销、分离处理中的完成批次、执行中历史页及内存后端保留消息另有占用。详细常量与生命周期见 docs/storage.md。

数据与代码的交付边界。. 本文源码及证明保存在 report/；构建 PDF、日志和页面预览保存在 .cache/report/。对外复现实验可以重新运行上述驱动；历史程序和原始 .cache 文件是本地证据，不会随 Git 克隆自动取得。表中完整数值、配置与匹配条件已在本文及长期文档保留。

B 顺序模型与机器检验

本附录把正文的房间顺序论证写成可检查的抽象状态机。Lean 4.33.1 使用标准库检验 report/formal/Ordering.lean；所有命题均有证明项，没有 sorry 或自设公理。模型解释单服务进程正常运行期间的新鲜成功发送，重试与失败不追加日志。它检验顺序机制，不替代 C++ 到模型的实现精化证明。

B.1 从提交到反应器处理的前缀关系

固定一个租户房间，将执行轨迹投影到新鲜成功发送，令状态

$$S = (C, A, D, P)$$

分别表示已提交 ID 日志、已进入完成队列的日志、反应器已处理的日志，以及提交后尚未进入完成队列的待入队事件。各日志元素为自然数，允许 ID 有洞；$X \preceq Y$ 表示列表前缀，$X \cdot Y$ 表示拼接。初态各列表为空。允许三种转移：

提交 commit(i)
仅当 $P = \epsilon$ 且对所有 $j \in C$ 有 $j < i$ 时，置 $C' = C \cdot [i]$、$P' = [i]$。

入队 admit(i)
仅当 $P = [i]$ 时，置 $A' = A \cdot [i]$、$P' = \epsilon$。

处理 drain(i)
仅当某个列表 T 满足 $A = D \cdot [i] \cdot T$ 时，置 $D' = D \cdot [i]$。

上述转移中未指明的分量保持不变。提交前的空待入队条件抽象了房间门控跨越提交至完成入队的连续性；严格增大条件来自房间锁下的存储 ID 分配，假设没有外部序列回退或迁移干预。

Proposition B.1 (可达状态不变量). *从初态经任意有限合法轨迹可达的状态均满足*

$$C = A \cdot P, \qquad D \preceq A, \tag{1}$$

$$|P| \le 1, \qquad C \text{ 严格递增}. \tag{2}$$

因此 $D \preceq C$，D 严格递增，且 $|C| \le |A| + 1$。

Proof. 初态直接满足。提交时 $P = \epsilon$，由旧等式得 $C = A$；两侧各追加同一新 ID，前缀和间隙约束保持，新鲜性保证严格递增。入队将唯一待入队 ID 从 P 移到 A 末尾，保持拼接等式且只延长 A。处理按条件只扩展 D 的下一个元素，因而保持其为 A 前缀。对轨迹长度归纳即得不变量；再用前缀传递、递增列表的前缀仍递增以及拼接长度性质，得到三个推论。对应 Lean 命题为 reachable_invariant、drained_prefix_committed、drained_ids_increase 和 admission_gap_at_most_one。 □

为什么数据库锁不够。. 若仅在数据库事务内加锁，允许

$$\text{commit}(1),\ \text{commit}(2),\ \text{admit}(2),\ \text{admit}(1).$$

此时提交日志为 $[1, 2]$，入队为 $[2, 1]$，后者不是前者前缀。Lean 的 reversed_admission_not_prefix 对这一具体反例作判定证明。它不证明任意无门控实现必然失败，而是说明仅有事务锁不能排除该轨迹。

模型到实现。. src/server/main.cpp 的 publication_lock 为房间选取锁带；schedule 的工作任务在 execute 前获取门控，直至 completed_.push_back 后才释放。PostgreSQL 的发送路径先锁房间、再分配并插入 ID，提交后返回；反应器按完成项 FIFO 处理。完整映射见 report/formal/README.md。这里 D 是反应器处理完成项，不是远端收包或用户显示；模型也不涵盖所有命令的线性一致性、进程崩溃后的发布或活性。

B.2 已读游标的代数性质

设旧游标为 r，请求为 x，房间最大 ID 为 M。存储先检查 $x \le M$，再更新

$$\text{advance}(r, x) = \max(r, x).$$

在 $r \le M$、$x \le M$ 前提下，有

$$r \le \text{advance}(r, x) \le M,$$

$$\text{advance}(\text{advance}(r, x), x) = \text{advance}(r, x),$$

$$\text{advance}(\text{advance}(r, x), y) = \text{advance}(\text{advance}(r, y), x).$$

Lean 的 cursor_monotone、cursor_bounded、cursor_retry_idempotent 和 cursor_updates_commute 分别检验单调性、有界性、重复幂等性和两次更新的交换性。前台布局决定允许提交哪个 x，代数性质仅保证存储更新行为；它不声称用户已看过全部较小 ID 的历史。ID 差也不代表消息数量。

B.3 检验命令与数值核算

在仓库根目录执行：

```
elan run leanprover/lean4:v4.33.1 lean report/formal/Ordering.lean
python report/check_metrics.py
python report/build.py
```

check_metrics.py 使用 SymPy 1.14.0 的有理数进行精确核算，不以浮点四舍五入后的比值再次推算百分比。以表 2 的三次试验中位数计算，吞吐比为 $2652.47/651.64$，显示为 4.07 倍；p50 降幅为 $1 - 5.568/24.466$，显示为 77.2%。终端突发中位数之比分别显示为 127.2 与 85.9 倍。这些是测量值的算术摘要，不是泛化性能定律。证明与算术的机器可执行文件分开维护，以免把实测估计误当作形式定理。